

PRAKTIKUM
SISTEM BASIS DATA

Nama	Nahl Syareza Rahidra	No. Modul	6
NPM	2206830340	Tipe	TUTAM

1. Oke siap lanjutkan
2. Berikut response yang telah ditambahkan sesuai format

Method	Endpoint	Success	Fail
GET	/transaction	<pre>{ "success": true, "message": "Transactions received!", "payload": [{ "id": "ef7bb7d8-a706-46ef-8516-5f885fe806db", "user_id": "b92dea99-701c-4a04-9289-b50f86175185", "item_id": "010a328a-aba6-45ef-9ff4-44439493bf6f", "quantity": 10, "total": 5000, "status": "pending", "created_at": "2025-04-10T05:33:59.196Z" }] }</pre>	<pre><!DOCTYPE html> <html lang="en"> <head> <meta charset="utf-8"> <title>Error</title> </head> <body> <pre>Cannot POST /transaction</pre> </body> </html></pre>

3. Di sini kita akan melakukan peningkatan skalabilitas dan beberapa error handling. Skalabilitas dapat ditingkatkan dengan melepas koneksi SQL. Ini bisa kita lakukan dengan cara meng-export pool dari pg.database.js. Contoh akan diberikan pada soal selanjutnya. Untuk error handling, kita bisa merancang repository untuk me-return value berupa string yang kemudian akan dicek pada controller menggunakan switch case. Contoh akan diberikan pada soal selanjutnya.

4. Pembuktian implementasi Scalability, Security, dan Error Handling

Type	Implementasi	Deskripsi
Scalability	<pre>exports.getTransaction = async (req) => { let client; try { client = await db.pool.connect(); const res = await client.query("SELECT * FROM transactions"); return res; } catch (e) { return e; } finally { if (client) { client.release(); } } };</pre>	Melepas koneksi ketika sudah menyelesaikan query agar tidak terjadi tumpukkan koneksi ke PsqL yang redundan
Security	<pre>const hashedPassword = await bcrypt.hash(data.password, saltLength); console.log(hashedPassword);</pre>	Mengimplementasikan hashing menggunakan bcrypt. Algoritma bcrypt dianggap sebagai salah satu algoritma yang aman dikarenakan ia menambahkan salt kepada password yang di-hash
Security	<pre>app.use(cors({ origin: "http://127.0.0.1/", methods: "GET,POST,PUT,DELETE", }));</pre>	Implementasi CORS untuk memastikan backend diakses dari IP yang dipercaya dengan method-method tertentu
Security	<pre>exports.emailRegex = (str) => { const emailRegex = /^[a-zA-Z0-9._-]+@[a-zA-Z0-9._-]+\.[a-zA-Z]{2,}\$/; return emailRegex.test(str); }; exports.passwordRegex = (str) => { const passwordSpecCharRegex = /[!@#%&*()_+~\[\]\{\};':"\ ,.<>\/?]{1}/; const passwordNumRegex = /[0-9]{1}/; return [data.password.length > 7 && passwordSpecCharRegex.test(data.password) && passwordNumRegex.test(data.password)]; };</pre>	Implementasi regex juga turut membantu dalam keamanan, khususnya regex password. Hal ini dicapai dengan membuat user perlu memiliki berbagai elemen pada password mereka, misalnya angka, huruf, karakter spesial, dan panjang password itu sendiri

Error Handling	<pre> switch (payTransaction) { case "blankFieldErr": return baseRes(res, false, 400, "All field must be filled!", null); case "transactionPaidErr": return baseRes(res, false, 200, "This transaction has already been paid!", null); case "lowBalanceErr": return baseRes(res, false, 200, "User have low balance", null); default: return baseRes(res, true, 200, "Success", payTransaction); } const transactRes = await client.query("SELECT * FROM transactions WHERE id=\$1", [req.id]); const transactStatus = transactRes.rows[0].status; if (transactStatus == "paid") { return "transactionPaidErr"; } const transactTotal = transactRes.rows[0].total; const userBal = userRes.rows[0].balance; if (transactTotal > userBal) { return "lowBalanceErr"; } </pre>	Ketika user melakukan query, terdapat beberapa kasus dimana input yang diberikan user bukan merupakan input yang diinginkan atau bahkan terdapat input esensial yang tidak dimasukkan oleh user. Di sini kita akan mengimplementasikan error handling. Error handling diimplementasikan dengan mengecek return value dari repository. Sebelum menjalankan query, kode dari repository akan mengecek beberapa kondisi. Jika terdapat check yang tidak lolos, maka kode akan mengembalikan string tergantung dengan tipe error. Ini kemudian dicek menggunakan switch case di controller, dengan default dianggap kondisi yang sukses
----------------	--	--

5. Oke siap bos dikumpulkan